04_excercise_run3

September 29, 2025

[1]:
```
from pathlib import Path
import numpy as np
import torch
from typing import List
from torch.nn.utils.rnn import pad_sequence
from mltrainer import rnn_models, Trainer
from torch import optim

from mads_datasets import datatools
import mltrainer
mltrainer.__version__
```

[1]:
```
'0.2.4'
```

1 1 Iterators

We will be using an interesting dataset. link

From the site: > The SmartWatch Gestures Dataset has been collected to evaluate several gesture recognition algorithms for interacting with mobile applications using arm gestures. Eight different users performed twenty repetitions of twenty different gestures, for a total of 3200 sequences. Each sequence contains acceleration data from the 3-axis accelerometer of a first generation Sony SmartWatch™, as well as timestamps from the different clock sources available on an Android device. The smartwatch was worn on the user's right wrist.

[2]:
```
from mads_datasets import DatasetFactoryProvider, DatasetType
from mltrainer.preprocessors import PaddedPreprocessor
preprocessor = PaddedPreprocessor()

gesturesdatasetfactory = DatasetFactoryProvider.create_factory(DatasetType.
 ↪GESTURES)
streamers = gesturesdatasetfactory.create_datastreamer(batchsize=32,␣
 ↪preprocessor=preprocessor)
train = streamers["train"]
valid = streamers["valid"]
```

```
2025-09-29 09:36:12.106 | INFO     |
```

```
mads_datasets.base:download_data:121 - Folder
already exists at /Users/nickreinders/.cache/mads_datasets/gestures
100%|      | 2600/2600 [00:00<00:00, 5463.39it/s]
100%|      | 651/651 [00:00<00:00, 5929.29it/s]
```

[3]:
```
len(train), len(valid)
```

[3]:
```
(81, 20)
```

[4]:
```
trainstreamer = train.stream()
validstreamer = valid.stream()
x, y = next(iter(trainstreamer))
x.shape, y
```

[4]:
```
(torch.Size([32, 34, 3]),
 tensor([14,  3, 13,  7,  0, 19,  1,  7,  5,  0, 19,  6, 19, 17, 13,  9,  8, 19,
         10,  4,  6,  4, 11,  8,  4, 19, 10, 14,  0,  1, 16, 10]))
```

Can you make sense of the shape? What does it mean that the shapes are sometimes (32, 27, 3), but a second time might look like (32, 30, 3)? In other words, the second (or first, if you insist on starting at 0) dimension changes. Why is that? How does the model handle this? Do you think this is already padded, or still has to be padded?

2 2 Excercises

Lets test a basemodel, and try to improve upon that.

Fill the gestures.gin file with relevant settings for `input_size`, `hidden_size`, `num_layers` and `horizon` (which, in our case, will be the number of classes…)

As a rule of thumbs: start lower than you expect to need!

[5]:
```
from mltrainer import TrainerSettings, ReportTypes
from mltrainer.metrics import Accuracy

accuracy = Accuracy()
```

[6]:
```
model = rnn_models.BaseRNN(
    input_size=3,
    hidden_size=64,
    num_layers=1,
    horizon=20,
)
```

Test the model. What is the output shape you need? Remember, we are doing classification!

[7]:
```
yhat = model(x)
yhat.shape
```

[7]:
```
torch.Size([32, 20])
```

Test the accuracy

[8]:
```
accuracy(y, yhat)
```

[8]:
```
0.03125
```

What do you think of the accuracy? What would you expect from blind guessing?

Check shape of `y` and `yhat`

[9]:
```
yhat.shape, y.shape
```

[9]:
```
(torch.Size([32, 20]), torch.Size([32]))
```

And look at the output of yhat

[10]:
```
yhat[0]
```

[10]:
```
tensor([-0.0137, -0.0430, -0.1326,  0.1033,  0.0757,  0.1040, -0.0032, -0.0168,
        -0.1990,  0.0546,  0.0852,  0.1230, -0.0027,  0.0403, -0.0379, -0.0454,
        -0.1991, -0.1454, -0.1328, -0.0905], grad_fn=<SelectBackward0>)
```

Does this make sense to you? If you are unclear, go back to the classification problem with the MNIST, where we had 10 classes.

We have a classification problem, so we need Cross Entropy Loss. Remember, this has a softmax built in

[11]:
```
loss_fn = torch.nn.CrossEntropyLoss()
loss = loss_fn(yhat, y)
loss
```

[11]:
```
tensor(3.0169, grad_fn=<NllLossBackward0>)
```

[12]:
```
import torch
if torch.backends.mps.is_available() and torch.backends.mps.is_built():
    device = torch.device("mps")
    print("Using MPS")
elif torch.cuda.is_available():
    device = "cuda:0"
    print("using cuda")
else:
    device = "cpu"
    print("using cpu")

# on my mac, at least for the BaseRNN model, mps does not speed up training
# probably because the overhead of copying the data to the GPU is too high
# so i override the device to cpu
device = "cpu"
```

```
# however, it might speed up training for larger models, with more parameters
```

```
Using MPS
```

Set up the settings for the trainer and the different types of logging you want

[13]:
```
settings = TrainerSettings(
    epochs=3, # increase this to about 100 for training
    metrics=[accuracy],
    logdir=Path("gestures"),
    train_steps=len(train),
    valid_steps=len(valid),
    reporttypes=[ReportTypes.TOML, ReportTypes.TENSORBOARD, ReportTypes.MLFLOW],
    scheduler_kwargs={"factor": 0.5, "patience": 5},
    earlystop_kwargs = {
        "save": False, # save every best model, and restore the best one
        "verbose": True,
        "patience": 5, # number of epochs with no improvement after which␣
 ↪training will be stopped
        "delta": 0.0, # minimum change to be considered an improvement
    }
)
settings
```

[13]:
```
epochs: 3
metrics: [Accuracy]
logdir: gestures
train_steps: 81
valid_steps: 20
reporttypes: [<ReportTypes.TOML: 'TOML'>, <ReportTypes.TENSORBOARD:
'TENSORBOARD'>, <ReportTypes.MLFLOW: 'MLFLOW'>]
optimizer_kwargs: {'lr': 0.001, 'weight_decay': 1e-05}
scheduler_kwargs: {'factor': 0.5, 'patience': 5}
earlystop_kwargs: {'save': False, 'verbose': True, 'patience': 5, 'delta': 0.0}
```

[14]:
```
import torch.nn as nn
import torch
from torch import Tensor
from dataclasses import dataclass

@dataclass
class ModelConfig:
    input_size: int
    hidden_size: int
    num_layers: int
    output_size: int
    dropout: float = 0.0
```

```
class GRUmodel(nn.Module):
    def __init__(
        self,
        config,
    ) -> None:
        super().__init__()
        self.config = config
        self.rnn = nn.GRU(
            input_size=config.input_size,
            hidden_size=config.hidden_size,
            dropout=config.dropout,
            batch_first=True,
            num_layers=config.num_layers,
        )
        self.linear = nn.Linear(config.hidden_size, config.output_size)

    def forward(self, x: Tensor) -> Tensor:
        x, _ = self.rnn(x)
        last_step = x[:, -1, :]
        yhat = self.linear(last_step)
        return yhat
```

[15]:
```
config = ModelConfig(
    input_size=3,
    hidden_size=64,
    num_layers=1,
    output_size=20,
    dropout=0.0,
)
```

[16]:
```
import mlflow
from datetime import datetime

mlflow.set_tracking_uri("sqlite:///mlflow.db")
mlflow.set_experiment("gestures")
modeldir = Path("gestures").resolve()
if not modeldir.exists():
    modeldir.mkdir(parents=True)

with mlflow.start_run():
    mlflow.set_tag("model", "modelname-here")
    mlflow.set_tag("dev", "your-name-here")
    config = ModelConfig(
        input_size=3,
        hidden_size=64,
        num_layers=1,
        output_size=20,
```

```
        dropout=0.1,
    )

    model = GRUmodel(
        config=config,
    )

    trainer = Trainer(
        model=model,
        settings=settings,
        loss_fn=loss_fn,
        optimizer=optim.Adam,
        traindataloader=trainstreamer,
        validdataloader=validstreamer,
        scheduler=optim.lr_scheduler.ReduceLROnPlateau,
        device=device,
    )
    trainer.loop()

    if not settings.earlystop_kwargs["save"]:
        tag = datetime.now().strftime("%Y%m%d-%H%M-")
        modelpath = modeldir / (tag + "model.pt")
        torch.save(model, modelpath)
```

```
/Users/nickreinders/Documents/Master Applied Data Science deeltijd
HAN/Deployment/Raoul_repo/MADS-MachineLearning-course/.venv/lib/python3.11/site-
packages/torch/nn/modules/rnn.py:123: UserWarning: dropout option adds dropout
after all but last recurrent layer, so non-zero dropout expects num_layers
greater than 1, but got dropout=0.1 and num_layers=1
  warnings.warn(
2025-09-29 09:36:13.299 | INFO     |
mltrainer.trainer:dir_add_timestamp:24 - Logging

to gestures/20250929-093613
2025-09-29 09:36:13.924 | INFO     |
mltrainer.trainer:__init__:68 - Found

earlystop_kwargs in settings.Set to None if you dont want earlystopping.
100%|      | 81/81 [00:00<00:00, 319.55it/s]
2025-09-29 09:36:14.242 | INFO     |
mltrainer.trainer:report:209 - Epoch 0 train

2.8591 test 2.5323 metric ['0.1172']
100%|      | 81/81 [00:00<00:00, 327.16it/s]
2025-09-29 09:36:14.517 | INFO     |
mltrainer.trainer:report:209 - Epoch 1 train

2.3459 test 2.2818 metric ['0.1766']
100%|      | 81/81 [00:00<00:00, 269.77it/s]
```

```
2025-09-29 09:36:14.847 | INFO     |
mltrainer.trainer:report:209 - Epoch 2 train

2.1848 test 2.1057 metric ['0.2359']
100%|      | 3/3 [00:00<00:00,  3.38it/s]
```

Try to update the code above by changing the hyperparameters.

To discern between the changes, also modify the tag mlflow.set_tag("model", "new-tag-here") where you add a new tag of your choice. This way you can keep the models apart.

[17]:
```
trainer.loop() # if you want to pick up training, loop will continue from the␣
↪last epoch
```

```
100%|      | 81/81 [00:00<00:00, 284.91it/s]
2025-09-29 09:36:15.166 | INFO     |
mltrainer.trainer:report:175 - Resuming epochs

from previous training at 3
2025-09-29 09:36:15.178 | INFO     |
mltrainer.trainer:report:209 - Epoch 3 train

2.0142 test 1.8953 metric ['0.2969']
100%|      | 81/81 [00:00<00:00, 272.45it/s]
2025-09-29 09:36:15.503 | INFO     |
mltrainer.trainer:report:209 - Epoch 4 train

1.7919 test 1.6293 metric ['0.4344']
100%|      | 81/81 [00:00<00:00, 343.62it/s]
2025-09-29 09:36:15.764 | INFO     |
mltrainer.trainer:report:209 - Epoch 5 train

1.5174 test 1.4785 metric ['0.4813']
100%|      | 3/3 [00:00<00:00,  3.32it/s]
```

[18]:
```
mlflow.end_run()
```

Excercises:

- try to improve the RNN model
- test different things. What works? What does not?
- experiment with either GRU or LSTM layers, create your own models. Have a look at `mltrainer.rnn_models` for inspiration.
- experiment with adding Conv1D layers. Think about the necessary input-output dimensions of your tensors before and after each layer.

You should be able to get above 90% accuracy with the dataset. Create a report of 1 a4 about your experiments.

vanaf hier dingen proberen

[19]:
```
class LSTMmodel(nn.Module):
    def __init__(self, config) -> None:
```

```
        super().__init__()
        self.config = config
        self.rnn = nn.LSTM(
            input_size=config.input_size,
            hidden_size=config.hidden_size,
            dropout=config.dropout,
            batch_first=True,
            num_layers=config.num_layers,
        )
        self.linear = nn.Linear(config.hidden_size, config.output_size)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x, _ = self.rnn(x)
        last_step = x[:, -1, :]      # neem laatste timestep
        yhat = self.linear(last_step)
        return yhat
```

[20]:
```
config = ModelConfig(
    input_size=3,        # 3 sensorkanalen (accelerometer)
    hidden_size=128,     # groter dan 64 proberen
    num_layers=2,        # meer lagen
    output_size=20,      # 20 klassen (gestures)
    dropout=0.2,         # beetje regularisatie
)
```

[21]:
```
with mlflow.start_run():
    mlflow.set_tag("model", "LSTM-128x2")
    mlflow.set_tag("dev", "nick")

    model = LSTMmodel(config=config)

    trainer = Trainer(
        model=model,
        settings=settings,
        loss_fn=loss_fn,
        optimizer=optim.Adam,
        traindataloader=trainstreamer,
        validdataloader=validstreamer,
        scheduler=optim.lr_scheduler.ReduceLROnPlateau,
        device=device,
    )
    trainer.loop()
```

```
2025-09-29 09:36:15.805 | INFO     |
mltrainer.trainer:dir_add_timestamp:24 - Logging

to gestures/20250929-093615
2025-09-29 09:36:15.806 | INFO     |
```

```
mltrainer.trainer:__init__:68 - Found

earlystop_kwargs in settings.Set to None if you dont want earlystopping.
100%|      | 81/81 [00:00<00:00, 84.39it/s]
2025-09-29 09:36:16.875 | INFO     |
mltrainer.trainer:report:209 - Epoch 0 train

2.6156 test 2.2597 metric ['0.2016']
100%|      | 81/81 [00:00<00:00, 81.96it/s]
2025-09-29 09:36:17.976 | INFO     |
mltrainer.trainer:report:209 - Epoch 1 train

2.0595 test 1.8960 metric ['0.2984']
100%|      | 81/81 [00:00<00:00, 82.51it/s]
2025-09-29 09:36:19.102 | INFO     |
mltrainer.trainer:report:209 - Epoch 2 train

1.6730 test 1.4568 metric ['0.4000']
100%|      | 3/3 [00:03<00:00,  1.10s/it]
```

[22]:
```
trainer.loop()


mlflow.end_run()
```

```
100%|      | 81/81 [00:01<00:00, 72.96it/s]
2025-09-29 09:36:20.321 | INFO     |
mltrainer.trainer:report:175 - Resuming epochs

from previous training at 3
2025-09-29 09:36:20.329 | INFO     |
mltrainer.trainer:report:209 - Epoch 3 train

1.4260 test 1.2942 metric ['0.4578']
100%|      | 81/81 [00:00<00:00, 81.19it/s]
2025-09-29 09:36:21.478 | INFO     |
mltrainer.trainer:report:209 - Epoch 4 train

1.2164 test 1.1264 metric ['0.5016']
100%|      | 81/81 [00:01<00:00, 74.58it/s]
2025-09-29 09:36:22.680 | INFO     |
mltrainer.trainer:report:209 - Epoch 5 train

1.0850 test 0.9770 metric ['0.5594']
100%|      | 3/3 [00:03<00:00,  1.19s/it]
```

we gaan hier conv1d toevoegen voor rnn

[23]:
```
@dataclass
class HybridConfig:
    input_size: int       # aantal features (hier: 3 accelerometer-kanalen)
    hidden_size: int      # hidden size van RNN
```

```
    num_layers: int       # aantal RNN-lagen
    output_size: int      # aantal klassen (20 gestures)
    conv_channels: int = 16  # filters van Conv1D
    kernel_size: int = 3     # filtergrootte van Conv1D
    dropout: float = 0.0     # dropout voor regularisatie
```

[24]:
```
class ConvLSTMmodel(nn.Module):
    def __init__(self, config: HybridConfig) -> None:
        super().__init__()
        self.config = config

        # Conv1D laag: zoekt korte patronen
        self.conv1 = nn.Conv1d(
            in_channels=config.input_size,      # input features (3␣
 ↪sensorkanalen)
            out_channels=config.conv_channels,  # aantal filters
            kernel_size=config.kernel_size
        )
        self.relu = nn.ReLU()

        # RNN laag (LSTM in dit voorbeeld)
        self.rnn = nn.LSTM(
            input_size=config.conv_channels,    # output van Conv1D = features␣
 ↪voor RNN
            hidden_size=config.hidden_size,
            num_layers=config.num_layers,
            dropout=config.dropout,
            batch_first=True,
        )

        # Lineaire laag naar klassen
        self.linear = nn.Linear(config.hidden_size, config.output_size)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # x: (batch, timesteps, features) → (B, T, F)
        # Conv1D verwacht (batch, channels, timesteps)
        x = x.permute(0, 2, 1)   # (B, F, T)

        x = self.conv1(x)        # (B, conv_channels, T_new)
        x = self.relu(x)

        # terug naar (B, T_new, conv_channels) voor RNN
        x = x.permute(0, 2, 1)

        x, _ = self.rnn(x)       # (B, T_new, hidden_size)
        last_step = x[:, -1, :]  # laatste timestep
        yhat = self.linear(last_step)  # (B, output_size)
```

```
        return yhat
```

[25]:
```
config = HybridConfig(
    input_size=3,        # accelerometer x,y,z
    hidden_size=128,     # groter dan 64
    num_layers=1,        # begin met 1 LSTM-laag
    output_size=20,      # 20 gestures
    conv_channels=16,    # aantal conv filters
    kernel_size=3,       # kernelgrootte
    dropout=0.2,         # beetje dropout
)
```

[26]:
```
with mlflow.start_run():
    mlflow.set_tag("model", "Conv1D-LSTM-16f-128h")
    mlflow.set_tag("dev", "nick")

    model = ConvLSTMmodel(config=config)

    trainer = Trainer(
        model=model,
        settings=settings,
        loss_fn=loss_fn,
        optimizer=optim.Adam,
        traindataloader=trainstreamer,
        validdataloader=validstreamer,
        scheduler=optim.lr_scheduler.ReduceLROnPlateau,
        device=device,
    )
    trainer.loop()
```

```
/Users/nickreinders/Documents/Master Applied Data Science deeltijd
HAN/Deployment/Raoul_repo/MADS-MachineLearning-course/.venv/lib/python3.11/site-
packages/torch/nn/modules/rnn.py:123: UserWarning: dropout option adds dropout
after all but last recurrent layer, so non-zero dropout expects num_layers
greater than 1, but got dropout=0.2 and num_layers=1
  warnings.warn(
2025-09-29 09:36:22.728 | INFO     |
mltrainer.trainer:dir_add_timestamp:24 - Logging

to gestures/20250929-093622
2025-09-29 09:36:22.728 | INFO     |
mltrainer.trainer:__init__:68 - Found

earlystop_kwargs in settings.Set to None if you dont want earlystopping.
  0%|          | 0/3 [00:00<?, ?it/s]2025-09-29

09:36:22.728 | INFO     |
mltrainer.trainer:__init__:68 - Found

earlystop_kwargs in settings.Set to None if you dont want earlystopping.
```

```
100%|      | 81/81 [00:00<00:00, 153.17it/s]
2025-09-29 09:36:23.342 | INFO     |
mltrainer.trainer:report:209 - Epoch 0 train

2.6811 test 2.3322 metric ['0.1688']
100%|      | 81/81 [00:00<00:00, 147.13it/s]
2025-09-29 09:36:23.956 | INFO     |
mltrainer.trainer:report:209 - Epoch 1 train

2.1099 test 2.0356 metric ['0.2875']
100%|      | 81/81 [00:00<00:00, 154.37it/s]
2025-09-29 09:36:24.541 | INFO     |
mltrainer.trainer:report:209 - Epoch 2 train

1.7015 test 1.5350 metric ['0.3891']
100%|      | 3/3 [00:01<00:00,  1.66it/s]
```

[27]:
```
trainer.loop()
```

```
100%|      | 81/81 [00:00<00:00, 149.29it/s]
2025-09-29 09:36:25.148 | INFO     |
mltrainer.trainer:report:175 - Resuming epochs

from previous training at 3
2025-09-29 09:36:25.156 | INFO     |
mltrainer.trainer:report:209 - Epoch 3 train

1.3605 test 1.3535 metric ['0.4703']
100%|      | 81/81 [00:00<00:00, 142.41it/s]
2025-09-29 09:36:25.803 | INFO     |
mltrainer.trainer:report:209 - Epoch 4 train

1.0507 test 0.9921 metric ['0.5656']
100%|      | 81/81 [00:00<00:00, 140.95it/s]
2025-09-29 09:36:26.446 | INFO     |
mltrainer.trainer:report:209 - Epoch 5 train

0.8393 test 0.8079 metric ['0.6828']
100%|      | 3/3 [00:01<00:00,  1.58it/s]
```

[28]:
```
trainer.loop()
```

```
100%|      | 81/81 [00:00<00:00, 157.31it/s]
2025-09-29 09:36:27.034 | INFO     |
mltrainer.trainer:report:175 - Resuming epochs

from previous training at 6
2025-09-29 09:36:27.039 | INFO     |
mltrainer.trainer:report:209 - Epoch 6 train

0.7069 test 0.6435 metric ['0.7750']
100%|      | 81/81 [00:00<00:00, 153.34it/s]
```

```
2025-09-29 09:36:27.635 | INFO     |
mltrainer.trainer:report:209 - Epoch 7 train

0.5368 test 0.5587 metric ['0.7984']
100%|      | 81/81 [00:00<00:00, 150.45it/s]
2025-09-29 09:36:28.241 | INFO     |
mltrainer.trainer:report:209 - Epoch 8 train

0.4038 test 0.3383 metric ['0.9094']
100%|      | 3/3 [00:01<00:00,  1.68it/s]
```

[29]:
```
trainer.loop()
```

```
  0%|          | 0/3 [00:00<?, ?it/s]
100%|      | 81/81 [00:00<00:00, 140.80it/s]
2025-09-29 09:36:28.896 | INFO     |
mltrainer.trainer:report:175 - Resuming epochs

from previous training at 9
2025-09-29 09:36:28.904 | INFO     |
mltrainer.trainer:report:209 - Epoch 9 train

0.3174 test 0.2213 metric ['0.9469']
100%|      | 81/81 [00:00<00:00, 159.69it/s]
2025-09-29 09:36:29.475 | INFO     |
mltrainer.trainer:report:209 - Epoch 10 train

0.2100 test 0.3190 metric ['0.9109']
2025-09-29 09:36:29.476 | INFO     |
mltrainer.trainer:__call__:252 - best loss:

0.2213, current loss 0.3190.Counter 1/5.
100%|      | 81/81 [00:00<00:00, 165.89it/s]
2025-09-29 09:36:30.026 | INFO     |
mltrainer.trainer:report:209 - Epoch 11 train

0.1660 test 0.2028 metric ['0.9484']
100%|      | 3/3 [00:01<00:00,  1.70it/s]
```

[30]:
```
trainer.loop()
```

```
100%|      | 81/81 [00:00<00:00, 160.83it/s]
2025-09-29 09:36:30.637 | INFO     |
mltrainer.trainer:report:175 - Resuming epochs

from previous training at 12
2025-09-29 09:36:30.644 | INFO     |
mltrainer.trainer:report:209 - Epoch 12 train

0.1546 test 0.2919 metric ['0.8953']
2025-09-29 09:36:30.644 | INFO     |
```

```
mltrainer.trainer:__call__:252 - best loss:

0.2028, current loss 0.2919.Counter 1/5.
100%|      | 81/81 [00:00<00:00, 139.91it/s]
2025-09-29 09:36:31.285 | INFO     |
mltrainer.trainer:report:209 - Epoch 13 train

0.1392 test 0.2109 metric ['0.9469']
2025-09-29 09:36:31.285 | INFO     |
mltrainer.trainer:__call__:252 - best loss:

0.2028, current loss 0.2109.Counter 2/5.
100%|      | 81/81 [00:00<00:00, 155.99it/s]
2025-09-29 09:36:31.866 | INFO     |
mltrainer.trainer:report:209 - Epoch 14 train

0.1825 test 0.1610 metric ['0.9641']
100%|      | 3/3 [00:01<00:00,  1.65it/s]
```

[31]:
```
trainer.loop()
```

```
100%|      | 81/81 [00:00<00:00, 162.37it/s]
2025-09-29 09:36:32.426 | INFO     |
mltrainer.trainer:report:175 - Resuming epochs

from previous training at 15
2025-09-29 09:36:32.432 | INFO     |
mltrainer.trainer:report:209 - Epoch 15 train

0.0941 test 0.0999 metric ['0.9797']
100%|      | 81/81 [00:00<00:00, 165.12it/s]
2025-09-29 09:36:32.983 | INFO     |
mltrainer.trainer:report:209 - Epoch 16 train

0.0688 test 0.0707 metric ['0.9875']
100%|      | 81/81 [00:00<00:00, 160.94it/s]
2025-09-29 09:36:33.546 | INFO     |
mltrainer.trainer:report:209 - Epoch 17 train

0.0690 test 0.0905 metric ['0.9719']
2025-09-29 09:36:33.546 | INFO     |
mltrainer.trainer:__call__:252 - best loss:

0.0707, current loss 0.0905.Counter 1/5.
100%|      | 3/3 [00:01<00:00,  1.79it/s]
```

[32]:
```
trainer.loop()
```

```
100%|      | 81/81 [00:00<00:00, 163.86it/s]
2025-09-29 09:36:34.109 | INFO     |
mltrainer.trainer:report:175 - Resuming epochs

from previous training at 18
```

```
2025-09-29 09:36:34.136 | INFO     |
mltrainer.trainer:report:209 - Epoch 18 train

0.0519 test 0.0569 metric ['0.9891']
100%|      | 81/81 [00:00<00:00, 173.84it/s]
2025-09-29 09:36:34.657 | INFO     |
mltrainer.trainer:report:209 - Epoch 19 train

0.0614 test 0.2216 metric ['0.9375']
2025-09-29 09:36:34.657 | INFO     |
mltrainer.trainer:__call__:252 - best loss:

0.0569, current loss 0.2216.Counter 1/5.
100%|      | 81/81 [00:00<00:00, 159.34it/s]
2025-09-29 09:36:35.224 | INFO     |
mltrainer.trainer:report:209 - Epoch 20 train

0.1075 test 0.0976 metric ['0.9750']
2025-09-29 09:36:35.224 | INFO     |
mltrainer.trainer:__call__:252 - best loss:

0.0569, current loss 0.0976.Counter 2/5.
100%|      | 3/3 [00:01<00:00,  1.79it/s]
```

[33]:
```
trainer.loop()
```

```
100%|      | 81/81 [00:00<00:00, 152.04it/s]
2025-09-29 09:36:35.838 | INFO     |
mltrainer.trainer:report:175 - Resuming epochs

from previous training at 21
2025-09-29 09:36:35.844 | INFO     |
mltrainer.trainer:report:209 - Epoch 21 train

0.0361 test 0.0558 metric ['0.9906']
100%|      | 81/81 [00:00<00:00, 149.24it/s]
2025-09-29 09:36:36.456 | INFO     |
mltrainer.trainer:report:209 - Epoch 22 train

0.0250 test 0.0314 metric ['0.9875']
100%|      | 81/81 [00:00<00:00, 144.71it/s]
2025-09-29 09:36:37.076 | INFO     |
mltrainer.trainer:report:209 - Epoch 23 train

0.0225 test 0.0353 metric ['0.9922']
2025-09-29 09:36:37.077 | INFO     |
mltrainer.trainer:__call__:252 - best loss:

0.0314, current loss 0.0353.Counter 1/5.
100%|      | 3/3 [00:01<00:00,  1.62it/s]
```

[34]:
```
trainer.loop()
```

```
100%|      | 81/81 [00:00<00:00, 150.10it/s]
2025-09-29 09:36:37.683 | INFO     |
mltrainer.trainer:report:175 - Resuming epochs

from previous training at 24
2025-09-29 09:36:37.690 | INFO     |
mltrainer.trainer:report:209 - Epoch 24 train

0.0186 test 0.0831 metric ['0.9781']
2025-09-29 09:36:37.690 | INFO     |
mltrainer.trainer:__call__:252 - best loss:

0.0314, current loss 0.0831.Counter 2/5.
100%|      | 81/81 [00:00<00:00, 134.82it/s]
2025-09-29 09:36:38.362 | INFO     |
mltrainer.trainer:report:209 - Epoch 25 train

0.0364 test 0.1459 metric ['0.9594']
2025-09-29 09:36:38.362 | INFO     |
mltrainer.trainer:__call__:252 - best loss:

0.0314, current loss 0.1459.Counter 3/5.
100%|      | 81/81 [00:00<00:00, 142.96it/s]
2025-09-29 09:36:38.995 | INFO     |
mltrainer.trainer:report:209 - Epoch 26 train

0.0246 test 0.0648 metric ['0.9812']
2025-09-29 09:36:38.996 | INFO     |
mltrainer.trainer:__call__:252 - best loss:

0.0314, current loss 0.0648.Counter 4/5.
100%|      | 3/3 [00:01<00:00,  1.57it/s]
```

[35]:
```
mlflow.end_run()
```